Crosswalk User Guide

AI-native career copilot · v0.3.0

Mohak Garg

May 2026

Welcome

Crosswalk is an AI-native career copilot. You install it once into your AI client (Claude Desktop, Cursor, etc.), and your AI gains 16 new tools to find jobs, score fit, tailor resumes, draft applications, track your pipeline, and run scheduled background workflows — all using the model you already pay for, with your data on your machine.

This guide walks you through installation, every tool, common workflows ("recipes"), troubleshooting, and frequently asked questions.

If you want the technical deep-dive (how it's built, the architectural bets, the data model), read `docs/ARCHITECTURE.md`. This document is the *user-facing* manual.

Part 1 — Getting started

1.1 Requirements

- **An MCP host that supports `sampling`.** The flagship is **Claude Desktop**. Cursor and Windsurf have partial support; ChatGPT MCP doesn't yet.
- **Node.js 24 LTS** or newer (Node 22+ probably works but isn't tested).
- **macOS, Linux, or Windows.** All three are supported; the install paths and cron commands differ slightly per OS.

You don't need an OpenAI key, an Anthropic key, or any provider account. Crosswalk runs through your AI client's existing model.

1.2 Install

```
npx crosswalk-mcp install
```

That's the whole installation. The command:

1. Finds your Claude Desktop config (e.g., `~/Library/Application Support/Claude/claude_desktop_config.json` on macOS).
2. Adds an `mcpServers.crosswalk-mcp` entry that runs Crosswalk via `npx`.
3. Prints the path it modified.

Restart Claude Desktop. You'll see "crosswalk-mcp" listed in the MCP servers panel.

Custom install paths

If you want state stored somewhere other than `~/.crosswalk/`:

```
CROSSWALK_HOME=/some/other/path npx crosswalk-mcp install
```

If you want to install from a local clone (for development):

```
git clone https://github.com/Mohakgarg5/crosswalk-mcp.git
cd crosswalk-mcp
npm install && npm run build
CROSSWALK_INSTALL_COMMAND=node node dist/cli.js install
```

Then edit `claude_desktop_config.json` so `args` points at your local `dist/cli.js`.

1.3 First session — the 4-minute setup

After restarting Claude Desktop, open a new chat and say:

> *"Set up my profile: I'm a PM with 2 years at Acme Corp. I want roles in NYC or remote, focused on AI infra or developer tools. Comp floor is $180K."*

Claude calls `setup_profile`. The result is stored in `~/.crosswalk/state.db`.

> *"Add my resume from ~/Documents/resume.pdf, label it 'Generic PM'."*

Claude calls `add_resume`. The PDF is parsed, structured, and stored.

> *"Find PM roles at H-1B sponsors."*

Claude calls `fetch_jobs` with `h1bSponsorOnly: true`. You get back a ranked list with company, title, location, and the H-1B confidence.

> *"Why am I a fit for the Stripe Payments role?"*

Claude calls `explain_fit` with that job's id. You get a markdown narrative: percentage estimate, strengths (citing your resume), gaps, and how to position the application.

You're set up. Everything from here is iteration.

Part 2 — The 16 tools

Each tool below has a one-line description, the input it accepts, the output it returns, and an example chat dialogue. Tools that need sampling (LLM work) are flagged.

2.1 Profile and resumes

`setup_profile` ⚡ uses sampling

Stores a structured profile from a free-form description. The LLM extracts name, headline, years of experience, skills, what you want, and notes — and saves them to SQLite.

Input	Output
`description: string`	`{ profile }`

You say: "Set up my profile. I'm a senior PM with 6 years at marketplace companies (Etsy, then Shopify), strong on growth and pricing. I want NYC, hybrid, $220K floor, must avoid pure ML/AI roles."

Crosswalk does: stores `{ name, headline, years_experience: 6, skills: [...], wants: { roles, locations, comp_min: 220000, must_avoid: ['ml', 'ai'] }, notes }`.

`add_resume` ⚡ uses sampling

Parses a resume from a path (`.txt`, `.md`, `.docx`, `.pdf`) or raw text and stores it under a label. You can store multiple resume versions and Crosswalk will pick the best one per JD.

Input	Output
`{ path?, rawText?, label }`	`{ id, label }`

You say: "Add my resume from `~/Documents/senior_pm_marketplaces.docx`, label it 'Marketplaces PM'."

Crosswalk does: extracts the DOCX text, asks your AI to structure it (skills, experiences, education, projects, highlights), saves it under `id`.

You say: "Now add `~/Documents/growth_pm.docx` as 'Growth PM'."

You now have two resume versions. When you tailor for a JD, the picker chooses whichever one fits better.

`list_resumes`

Lists all stored resumes with id, label, and creation timestamp.

Input	Output
`{}`	`{ resumes: [{ id, label, createdAt }] }`

You say: "Which resumes do I have?"

Crosswalk does: returns the list. You see "Marketplaces PM" and "Growth PM" with their IDs.

2.2 Discovery and matching

`fetch_jobs`

Pulls live jobs from all 8 ATSs in parallel, filters by your criteria, and persists them to a local cache.

Input	Output
`{ titleContains?, locationContains?, remoteOnly?, sinceDays?, companyIds?, h1bSponsorOnly?, h1bMinConfidence?, limit? }`	`{ jobs: [...], meta: { fetched, afterFilters, companiesQueried, errors } }`

You say: "Find product manager roles posted in the last 7 days at H-1B sponsors with confidence above 0.85."

Crosswalk does: queries ~25 sponsor-flagged companies' ATSs, returns 30 ranked jobs. The `meta` field tells you how many companies it queried and which (if any) errored.

You say: "Find remote engineering roles at OpenAI, Anthropic, and Vercel."

Crosswalk does: filters to those 3 `companyIds`, applies `remoteOnly`, returns all matching engineering roles.

`score_fit` ⚡ uses sampling

Scores how well a stored resume fits a specific job. Returns a numeric score (0–1) plus structured strengths and gaps.

Input	Output
`{ jobId, resumeId? }`	`{ score, topStrengths: string[], topGaps: string[] }`

You say: "Score me against the Stripe Payments PM role."

Crosswalk does: sends your most recent resume + the JD to your AI, gets back `{ score: 0.78, topStrengths: ["6 years marketplace PM", "growth + pricing"], topGaps: ["no payments domain", "no Stripe API exposure"] }`.

If you have multiple resumes and want to score against a specific one: *"Score me against the Stripe role using my Marketplaces PM resume."*

`explain_fit` ⚡ uses sampling

Same data as `score_fit` but in narrative form — the markdown brief you'd skim before deciding to apply.

Input	Output
`{ jobId, resumeId? }`	`{ narrativeMd }`

You say: "Why am I a fit for that Stripe role?"

Crosswalk does: returns markdown like:

> *78% fit — strong PM-domain match but light on payments specifics.*
>
> ***Strengths.*** *Six years of marketplace PM at Etsy and Shopify; payments adjacency through Shopify Payments cross-functional work; quantified pricing wins ("raised conversion 12% via dynamic pricing").*
>
> ***Gaps.*** *No direct Stripe API or fraud experience. Not deep on enterprise sales motion.*
>
> ***Positioning.*** *Lead the cover letter with the Shopify Payments cross-functional work; treat the gaps as upside ("eager to specialize in payments depth").*

2.3 Authoring

`tailor_resume` ⚡ uses sampling

Edits your best base resume for a specific JD. Returns markdown by default; can additionally produce a DOCX (base64-encoded) or print-styled HTML.

Input	Output
`{ jobId, resumeId?, format? }`	`{ tailoredMd, resumeId, pickedReason, docxBase64?, html? }`

`format` accepts `'md'` (default), `'docx'`, or `'html'`.

You say: "Tailor my resume for the Stripe role and give me the DOCX."

Crosswalk does: picks your best base resume (Marketplaces PM), tailors it (adds payments keywords where your facts justify, reorders bullets, drops irrelevant ones), and returns the markdown plus a base64-encoded DOCX. Save the DOCX with:

```
echo "<base64 string>" | base64 --decode > tailored_resume.docx
```

You say: "Same thing but as printable HTML."

Crosswalk does: returns a complete HTML document with print CSS. Open it in your browser, hit "Save as PDF."

`draft_application` ⚡ uses sampling, has guardrail

Builds a complete "application PR": picks the best resume, tailors it for the JD, drafts a cover letter, persists everything as a tracked draft application, and returns the bundle.

Input	Output
`{ jobId, resumeId?, allowDuplicate?, confirmLowFit? }`	`{ applicationId, tailoredResumeMd, coverLetterMd, deepLink, ... }`

You say: "Draft an application for the Stripe role."

Crosswalk does: runs the guardrail (weekly cap + dup check). If allowed, picks your best resume, tailors it, drafts a 200–300 word cover letter, persists as a `draft` application, returns the bundle including the deep link to apply.

If the guardrail refuses: you'll see something like *"already drafted an application (app_abc123, status=draft) for this job. Pass allowDuplicate=true to override."* You can override: *"Draft it anyway."*

2.4 Pipeline tracker

`submit_application`

After you've actually clicked "Apply" on the company's site, mark the application submitted. This stamps the timestamp and logs a `status_changed` event.

Input	Output
`{ applicationId }`	`{ applicationId, status: 'submitted', submittedAt }`

You say: "I just submitted the Stripe app — mark it done."

Crosswalk does: updates the row, logs the event.

`set_status`

For arbitrary status transitions: a recruiter responded → `interviewing`; you got a rejection → `rejected`; you got an offer → `offer`.

Input	Output
`{ applicationId, status: 'draft' | 'submitted' | 'interviewing' | 'rejected' | 'offer' }`	`{ applicationId, status }`

You say: "The Stripe recruiter just emailed me — move that to interviewing."

Crosswalk does: updates status, logs `{ kind: 'status_changed', from: 'submitted', to: 'interviewing' }`.

`add_note`

Append a free-text note to an application's event log. Use this for recruiter messages, interview notes, follow-up reminders.

Input	Output
`{ applicationId, text }`	`{ eventId }`

You say: "Add a note to the Stripe app: recruiter said the hiring manager wants to schedule a 30-min call next week."

`list_pipeline`

Lists all your applications with company + job context. Optionally filtered by status.

Input	Output
`{ status? }`	`{ items: [{ applicationId, status, jobTitle, company, deepLink, createdAt, submittedAt? }] }`

You say: "Show me my interviewing pipeline."

Crosswalk does: filtered query, returns just the `interviewing` rows. You see Stripe (PM Payments), Anthropic (PM Claude), Vercel (Senior PM).

You say: "Show everything."

Crosswalk does: all 14 applications across `draft`, `submitted`, `interviewing`, `rejected`, `offer`.

2.5 Scheduling

`schedule_workflow`

Schedules a recurring **non-sampling** workflow. Currently two kinds:

- `fetch_jobs_refresh` — re-runs `fetch_jobs` with stored params, refreshes the cache.
- `prune_old_jobs` — deletes cached jobs older than `params.olderThanDays`.

Input	Output
`{ kind, cron, description, params }`	`{ workflowId, nextRunAt }`

`cron` is a standard 5-field cron expression: `minute hour day-of-month month day-of-week`.

You say: "Schedule a workflow to refresh PM jobs at H-1B sponsors every Monday at 9 AM."

Crosswalk does: creates a `fetch_jobs_refresh` workflow with cron `0 9 * * 1` and params `{ titleContains: 'PM', h1bSponsorOnly: true }`. Returns the workflow id and `nextRunAt`.

To actually trigger workflows automatically, add this single line to your crontab (`crontab -e`):

```
* * * * * /usr/local/bin/crosswalk-mcp run-scheduled >> ~/.crosswalk/scheduler.log 2>&1
```

Every minute, your machine checks for due workflows and runs them. The log file shows what ran.

`run_workflow`

Manually trigger a scheduled workflow now (without waiting for cron).

Input	Output
`{ workflowId }`	`{ workflowId, status, summary?, nextRunAt }`

You say: "Run the H-1B refresh workflow now."

Crosswalk does: runs it immediately, updates `last_run_at` and `next_run_at`.

`list_workflows`

Lists all scheduled workflows with their last-run status.

Input	Output
`{}`	`{ workflows: [{ id, kind, description, cron, nextRunAt, lastRunAt?, lastStatus?, lastError? }] }`

You say: "What workflows do I have scheduled?"

`delete_workflow`

Deletes a scheduled workflow by id. Use this when you've changed your mind about a recurring job.

Input	Output
`{ workflowId }`	`{ deleted: boolean }`

Part 3 — Recipes (common workflows)

Recipe 1 — The Sunday session

Spend 30 minutes on Sunday refreshing your pipeline.

```
You: List my pipeline.
Crosswalk: [shows 9 active applications across statuses]

You: For the 3 in submitted state with no recruiter contact in 7+ days, draft polite
     follow-up notes I can paste into LinkedIn.
Crosswalk: [drafts 3 short LinkedIn DMs referencing the role + something specific
           from the JD; saves each as a note on the application]

You: Find new senior PM roles at H-1B sponsors posted this week.
Crosswalk: [returns 12 fresh roles, ranked by H-1B confidence]

You: Score me against each. Show me the top 5.
Crosswalk: [scores all 12, returns top 5 with explain_fit narratives]

You: Draft applications for the top 3.
Crosswalk: [runs the guardrail — caps at 7 this week, so all 3 pass.
           Drafts and persists each. Returns 3 bundles with deep links.]
```

You spend the next hour reviewing the 3 drafts in your editor, polishing where needed, then submitting via the deep links. Mark each `submit_application` after submitting. Done.

Recipe 2 — Targeted outreach

You're specifically interested in 3 companies and want a tight watching pattern.

```
You: Schedule a workflow to refresh jobs at Stripe, Anthropic, and Vercel
     every weekday at 8 AM.
Crosswalk: [creates a fetch_jobs_refresh workflow with companyIds:
           ['stripe','anthropic','vercel'], cron '0 8 * * 1-5']

You: List my workflows.
Crosswalk: [shows the new one + any others; nextRunAt at tomorrow 8 AM]
```

Then you add the cron line once. Every weekday morning your local cache has the freshest jobs from those 3 companies; when you next open Claude, asking *"What's new at Stripe?"* gets you the morning's listings without any waiting.

Recipe 3 — Multi-resume strategy

You have 3 distinct profiles you want to keep separate (e.g., PM, EM, Founder).

```
You: Add ~/resumes/pm.docx as 'PM track'.
You: Add ~/resumes/em.docx as 'EM track'.
You: Add ~/resumes/founder.docx as 'Founder track'.
You: List my resumes.
Crosswalk: [3 entries]

You: Tailor my resume for the Anthropic Research PM role.
Crosswalk: [picks 'PM track' automatically, tailors it. The pickedReason
           field tells you why: "PM track leads with research-oriented
           bullets and matches the JD's research focus better than EM track."]

You: Tailor my resume for the Linear founding engineering manager role
     using the EM track resume.
Crosswalk: [respects the explicit resumeId, skips the picker]
```

The picker runs once per draft and is cheap (one short sampling call). For a session where you draft 5 applications, you save 5 manual choices.

Recipe 4 — Pipeline review for a recruiter chat

A recruiter asks “where are you in your search?” and you want a one-screen summary.

```
You: Show my pipeline grouped by status.
Crosswalk: [returns counts: 6 drafts, 4 submitted, 3 interviewing, 1 offer, 2 rejected]

You: For the interviewing ones, summarize where I am with each.
Crosswalk: [for each interviewing app, pulls the event log via list_pipeline,
           summarizes the recent notes: "Stripe — recruiter call Mon 5/4,
           hiring manager Wed", etc.]
```

You paste the summary directly into the recruiter chat. Five minutes vs. an hour digging through email.

Recipe 5 — H-1B-focused search

You’re on OPT and need to filter aggressively for sponsors.

```
You: Set my profile to filter only H-1B sponsors with 0.8+ confidence by default.
Crosswalk: [updates the profile with want.must_have including h1b sponsorship]

You: Find me PM roles in NYC or remote.
Crosswalk: [applies your stored preferences automatically — h1bSponsorOnly:true,
           h1bMinConfidence:0.8 — returns 18 sponsor-confident roles]
```

The H-1B confidence values come from a public USCIS dataset checked into the repo at `registry/h1b.json`. They're heuristic — Crosswalk surfaces them rather than gating on them, so you can override per-search.

Recipe 6 — Cleaning up after a successful search

You accepted an offer. Crosswalk has 50+ applications stored. You want to keep history but stop the noise.

```
You: Set the offer status on the Stripe application.
Crosswalk: [updates, logs event]

You: Delete the H-1B refresh workflow.
Crosswalk: [removes it]

You: List my workflows.
Crosswalk: [empty]
```

Your `~/.crosswalk/state.db` keeps the full history — useful for the next search 18 months from now. If you want to start completely fresh: `rm ~/.crosswalk/state.db` (Crosswalk recreates it on next start).

Part 4 — Troubleshooting

“Crosswalk isn’t showing up in Claude Desktop”

1. Check that the install command actually ran: `cat ~/Library/Application\ Support/Claude/claude_desktop_config.json` (macOS) — look for the `crosswalk-mcp` entry under `mcpServers`.
2. Restart Claude Desktop fully (Cmd-Q, not just close-window).
3. Check the MCP servers panel in Claude Desktop’s settings. If it lists Crosswalk but with an error, click for details — the error message usually says exactly what’s missing (Node version, missing files, etc.).

“I get ‘unknown ats’ errors”

This means a company in your registry references an ATS slug that isn’t registered. Make sure you have v0.3.0 (`crosswalk-mcp --version`) — earlier versions only support 3 ATSs.

If you’re running locally from a clone, run `npm run build` after pulling.

“fetch_jobs returns errors for some companies”

Some companies have public ATS endpoints; some don’t. The `meta.errors` field of the result tells you which ones failed. Common causes:

- The company changed its ATS slug (file an issue or PR `registry/companies.json`).
- The company moved to a different ATS entirely (same fix).
- The company’s board returned a 5xx — try again in a few minutes.

You can run with `companyIds` to skip the problematic ones for a session.

“tailor_resume / score_fit / explain_fit hangs or times out”

These tools use MCP sampling, which talks to your AI host. If your AI client is offline, has hit a rate limit, or is mid-streaming a different response, the call can hang. Cancel the chat, wait a moment, and retry.

If you see the call going through but the response is empty, your AI host may have rejected the prompt for length. The picker uses 4000 chars of JD; tailorer uses 6000. Most JDs fit; a few don't. (We're aware; better truncation is on the M5 list.)

"schedule_workflow says my cron is invalid"

Crosswalk validates cron expressions via the `cron-parser` library. Standard 5-field cron (`minute hour dom month dow`) is supported. Aliases like `@daily` aren't. Examples:

You want	Cron expression
Every Monday at 9 AM	`0 9 * * 1`
Weekdays at 6 AM	`0 6 * * 1-5`
Every hour	`0 * * * *`
Every 15 minutes	`*/15 * * * *`
First of every month at midnight	`0 0 1 * *`

"run-scheduled doesn't actually run my workflows"

You scheduled a workflow but it's not firing. Things to check:

1. Did you add the crontab entry? `crontab -l` should show the line: `* * * * * /usr/local/bin/crosswalk-mcp run-scheduled >> ~/.crosswalk/scheduler.log 2>&1`
2. Is `crosswalk-mcp` actually in `/usr/local/bin/`? `which crosswalk-mcp` will tell you. If installed via `npx`, the path is different — use the full path.
3. Tail the log: `tail -f ~/.crosswalk/scheduler.log`. If cron is running it, you'll see output every minute.
4. macOS only: cron may need Full Disk Access permission to read `~/.crosswalk/`. System Settings → Privacy & Security → Full Disk Access → add `cron`.

"I want to nuke everything and start over"

```
rm -rf ~/.crosswalk
crontab -l | grep -v crosswalk-mcp | crontab -
```

Then restart Claude Desktop. Next time you talk to Crosswalk, the database is recreated empty and the company registry re-seeds.

Part 5 — Frequently asked questions

Q: Does Crosswalk send my resume anywhere?

A: To your AI host (Claude Desktop), which sends it to whichever model you've configured. The resume text doesn't go to any Crosswalk-operated server, because Crosswalk doesn't operate any servers. It runs locally as an MCP process.

Q: Where is my data stored?

A: One file: `~/.crosswalk/state.db`. SQLite. You can `git`-version it, back it up, copy it to another machine, or delete it.

Q: Why doesn't Crosswalk include Workday or iCIMS?

A: Neither has a public JSON endpoint that works for arbitrary employers. Building reliable scrapers for them needs a Playwright sandbox, which is M5. For now, paste a Workday/iCIMS JD into the chat and ask Claude to score it manually — you don't get the full draft_application loop, but you get fit reasoning.

Q: Can I use Crosswalk with ChatGPT?

A: Partial. ChatGPT's MCP support doesn't yet include `sampling`, so the AI-driven tools (tailor, draft, score) don't work. You can still use the read-only tools (`fetch_jobs`, `list_pipeline`, `list_resumes`).

Q: What's the deal with the H-1B confidence values?

A: They're heuristic, derived from the public USCIS H-1B Employer Data Hub (FY2025 snapshot). Confidence ≥ 0.8 means the company has been a meaningful sponsor recently; below that, treat it as a guess. The values are checked into `registry/h1b.json` so anyone can audit or correct them.

Q: Can I add my company to the registry?

A: Yes. PR `registry/companies.json` with your entry. The registry is MIT-licensed; Crosswalk just consumes it.

Q: Is there a hosted version?

A: Not in v0.3. The same code could deploy as a remote MCP on Vercel + Neon for non-dev users; that's an option for v0.4+ if there's demand.

Q: How is Crosswalk different from auto-apply tools?

A: Auto-apply tools fire 100–500 applications a day. Crosswalk caps you at 10 quality submissions per week and refuses duplicates. We're trying to make the job market less spam-y, not more.

Q: How is Crosswalk different from jobos.us?

A: jobos.us is a web platform: signup required, your data on their cloud, their AI key (Gemini), $19/mo. Crosswalk is an MCP server: no signup, your data on your machine, your AI key (whatever you already pay for), $0.

Q: Can I extend it?

A: Yes. Add a new ATS adapter in `src/ats/`, register in `src/server.ts` and `src/tools/fetch_jobs.ts`, send a PR. Or add a new MCP tool in `src/tools/` and register it in `src/tools/index.ts`. The architecture doc has a step-by-step.

Q: What are the known limitations?

A: As of v0.3.0: - No Workday or iCIMS support (M5). - The "low fit" guardrail gate is reserved but not yet active (M5). - Sampling-driven workflows (e.g., "tailor the top 3 fits weekly") aren't supported because workflows can't access the AI host while it's closed. - DOCX exports are good for typical resumes but lack inline formatting (bold, italic, links inside paragraphs). - The community registry sits at 51 companies; growing to 200+ is M5+ via PRs.

Part 6 — Reference

6.1 Environment variables

Variable	Purpose	Default
`CROSSWALK_HOME`	Where state.db lives	`~/.crosswalk`
`CROSSWALK_INSTALL_COMMAND`	Override the `command` written to Claude Desktop config	`npx`

6.2 CLI subcommands

Command	Purpose
`crosswalk-mcp` (no args)	Run as MCP server (used by Claude Desktop)
`crosswalk-mcp install`	Add Crosswalk to Claude Desktop config
`crosswalk-mcp run-scheduled`	Run any due workflows now (called from cron)
`crosswalk-mcp --version`	Print version (e.g. `0.3.0`)
`crosswalk-mcp --help`	Show usage

6.3 ATS coverage at v0.3

ATS	Companies in seed registry
Greenhouse	Stripe, Airbnb, Discord, Anthropic, Vercel, Figma, Linear, Instacart, DoorDash, Robinhood, Scale AI, Coinbase, Asana, Datadog, Snowflake (15)
Lever	Netflix, Spotify, Shopify, Brex, Lyft, GitHub, Checkr, Kraken (8)
Ashby	OpenAI, Ramp, Hex, Deel, Notion, Browserbase, Modal Labs (7)
Workable	Miro, n8n, Remote.com, Deepfence (4)
SmartRecruiters	Bosch, Siemens, Ubisoft, OpenText (4)
BambooHR	Klaviyo, Buffer, Zapier, Tinybird (4)
Recruitee	Mollie, MessageBird, HelloFresh, The Trade Desk (4)
Personio	Personio, Clue, Trade Republic, Scalable Capital (4)

Total: 51 companies, 8 ATSs. Expanding by community PR.

6.4 The 16 tools at a glance

#	Tool	Sampling?	Purpose
1	`setup_profile`	⚡	Store profile from free-form description
2	`add_resume`	⚡	Parse + store a labeled resume
3	`list_resumes`		List stored resumes
4	`fetch_jobs`		Pull live jobs across 8 ATSs
5	`score_fit`	⚡	Numeric fit score + structured strengths/gaps
6	`explain_fit`	⚡	Markdown narrative on fit
7	`tailor_resume`	⚡	Edit base resume for a JD; md/docx/html
8	`draft_application`	⚡	Build complete application PR (resume + cover letter + deep link)
9	`submit_application`		Mark application submitted; stamp timestamp
10	`set_status`		Change application status
11	`add_note`		Append free-text note to event log
12	`list_pipeline`		List applications with company + job context
13	`schedule_workflow`		Schedule recurring workflow via cron

#	Tool	Sampling?	Purpose
14	`run_workflow`		Manually run a scheduled workflow now
15	`list_workflows`		List scheduled workflows
16	`delete_workflow`		Delete a scheduled workflow

6.5 Privacy summary

- **Data storage:** local SQLite at `~/.crosswalk/state.db`.
- **Network calls:** only ATS APIs (when you call `fetch_jobs`) and your AI host (for sampling). Nothing else.
- **Telemetry:** none.
- **Removal:** `rm -rf ~/.crosswalk` and remove from Claude Desktop config.

6.6 Project links

Resource	URL
GitHub repo	https://github.com/Mohakgarg5/crosswalk-mcp
README	https://github.com/Mohakgarg5/crosswalk-mcp/blob/main/README.md
Architecture doc	https://github.com/Mohakgarg5/crosswalk-mcp/blob/main/docs/ARCHITECTURE.md
Implementation plans	https://github.com/Mohakgarg5/crosswalk-mcp/tree/main/docs/superpowers/plans
npm package	https://www.npmjs.com/package/crosswalk-mcp
File issues	https://github.com/Mohakgarg5/crosswalk-mcp/issues

6.7 Roadmap snapshot

Version	Status	Theme
v0.0.1	Shipped	M1 — Discover + match + explain
v0.1.0	Shipped	M2 — Tailor + draft application
v0.2.0	Shipped	M3 — Pipeline tracker + guardrail + workflows
v0.3.0	**Current**	**M4 — Reach (8 ATSs, 51 companies)**
v0.4.0	Next	M5 — Workday + iCIMS via Playwright; live-fit gate; registry to 200+
v1.0.0	Planned	v2 — Autonomous apply via Playwright sandbox

Appendix — License and credits

Crosswalk is **MIT-licensed**. Use it, fork it, build on top of it, ship it commercially — just keep the license notice.

Built with Claude Code (Opus 4.7, 1M context). Every commit on the project is co-authored by the model that wrote it.

Inspired by: `jd-intel` — the LinkedIn-post MCP that proved the "fetch jobs from your AI" hook works.

H-1B data: derived from the publicly-published USCIS H-1B Employer Data Hub (FY2025 snapshot).

Issues & contributions: https://github.com/Mohakgarg5/crosswalk-mcp/issues.

End of Crosswalk User Guide v0.3.0.